

# Managing Change With the Funnel Model of Agile UX

## Highlights

- The challenges of managing change in a lifecycle process
  - The inevitability of change
  - Managing the state of designer knowledge of requirements and design
  - The role of feedback and learning in closing the gaps in requirements knowledge
- The old waterfall lifecycle model
  - The feedback mechanisms it had, and why they were not the right kind
- Embracing an agile lifecycle process
  - How scope and chunking are the most important characteristics
  - How agility is an outcome of chunking
  - Agile user experience design (UX)
  - Agile software engineering (SE)
- Looking at change in terms of divergence between understanding and reality
- The funnel model of agile UX
  - Early-funnel activities: upfront analysis and design
  - Late-funnel activities: syncing with the SE sprints

## 4.1 THE CHALLENGES OF MANAGING CHANGE IN DESIGNING AND BUILDING SYSTEMS

### 4.1.1 Change Happens During a Project

Perhaps the biggest single lesson learned in the history of software engineering (and UX design) is that change is inevitable (Sommerville, 2015). Your lifecycle process not only must handle change, but it also must embrace change and incorporate it as an operational feature and a necessary mechanism for learning throughout the process (Dubberly & Evenson, 2011).

### 4.1.2 Almost Everything May Change During an Iteration of the Lifecycle Process

Many things may change during an iteration of the lifecycle process, including:

- Design requirements can evolve because of what you learned from design evaluation and feedback (see [Part 5](#)).
- Project parameters (e.g., project scope, personnel, budget, project vision) can mature with time and exposure to the team.
- The product concept and project vision can get clearer as design work progresses.
- The client's direction and focus can change because of shifting organizational goals, market factors, or government regulations.
- Conceptualization of system architecture can evolve as we learn more about the requirements.
- Design ideas can become more focused and more feasible.
- Available technology can change, often because of paradigm shifts.

### 4.1.3 Getting Timely Feedback Is the Key to Understanding Change

Because of these changes that occur during each iteration of the lifecycle process, UX design (and software engineering [SE] implementation) chases a moving target. If you have a good process, your knowledge of what will turn out to be the most appropriate requirements will improve over these iterations. Because of this learning due to change, the end of a lifecycle iteration is an appropriate time to get feedback from the following:

- Actual usage, which is the only way to help designers understand real requirements
- The client or sponsor, with respect to product concept and vision
- Designers, on their retrospective view of the design so far
- Software and system engineering people, about possible needed changes in system architecture

The lack of timely feedback is the reason why old approaches often failed. The team invested too much before seeing any results. They were not able to see when they needed a course correction.

---

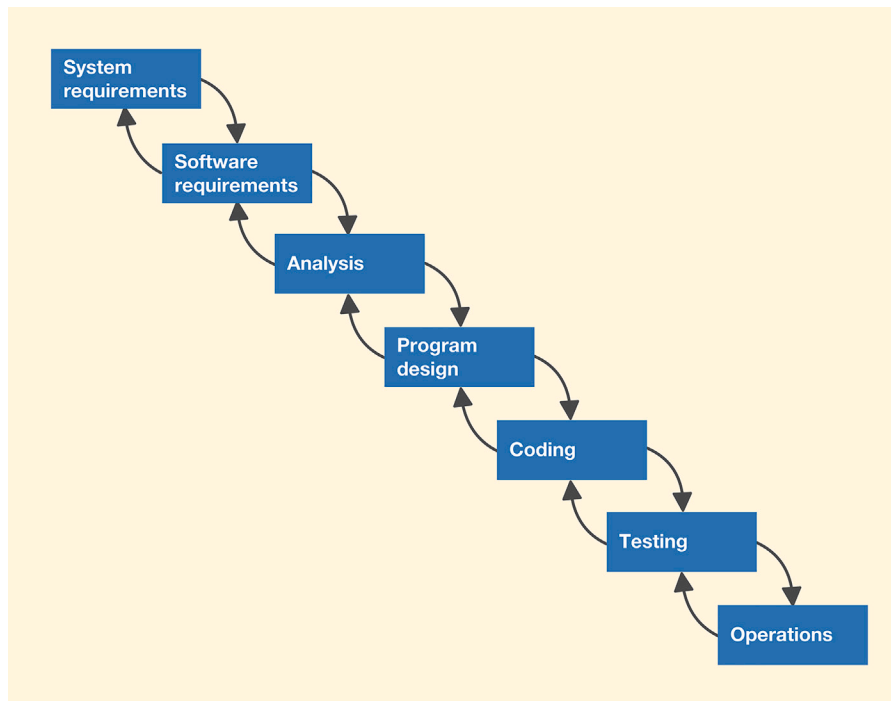
## 4.2 THE WATERFALL MODEL OF SOFTWARE SYSTEM DEVELOPMENT

The old waterfall model of software system development was well known for its inability to manage change. The original waterfall model was developed in an

era when most systems were large enterprise systems. Users were trained to use a system for specific business purposes, and there was little consideration of usability or UX.

The first mention of the waterfall model we could find is a presentation by Bennington (1956) that was later republished in a historical perspective (Benington, 1983). Bennington added a foreword stating that the process was not intended to be used in a strict top-down way and that it depended on a prototype phase. Royce (1970) is the most frequently cited source and probably the first formal description of the process. The method had already been in use for some time, however, and the Royce paper seems to be less intended as a prescriptive model and more as a critique of the method that failed. It was not called the waterfall model, however, until 1976 (Bell & Thayer, 1976).

The waterfall model is one of the simplest (in form, at least) ways to put lifecycle activities together to make the SE lifecycle process. The process was so named because it was described as an ordered, essentially linear sequence of phases (lifecycle activities), each of which flowed into the next like a set of cascading tiers of a waterfall (Fig. 4.1). Each lifecycle activity, in turn, is performed for the entire system scope.



*Fig. 4.1  
Cascades from phase to  
phase in the waterfall model  
(adapted from Royce (1970)).*

A strong underlying principle (as typically practiced) stated that work on any phase was not allowed until all previous phases were completed. The objective of the waterfall model was to deliver the full system at once at the end of the full lifecycle. This process was an attempt to overcome previous relatively undisciplined approaches to software, so it was methodical and mostly rigorous. Because it operated with large scope in the extreme, however, the waterfall model was slow, cumbersome, unmanageable, and not responsive to change. When problems were inevitably discovered at the end of each phase, it was usually too expensive at that point to go back and fix the problems in all the previous phases of the whole system scope. The long and drawn-out lifecycle of the waterfall model proved unsustainable, resulting in projects that were over budget and behind schedule. It was impossible to manage change, and many projects collapsed under their own weight.

The culture of this era was that of a very long product or system shelf life because making changes was so difficult. Project goals were missed, some requirements were wrong, other requirements went unmet, the system had severe usability problems, and the final deliverable failed to satisfy its intended purpose.

In 1985, the Department of Defense decided (unfortunately, in retrospect) to make the waterfall approach a software development standard (Department of Defense, 1998) for their software development contractors. With the advent of agile software development, the waterfall model is no longer in popular use.

---

## 4.3 EMBRACING AN AGILE SE LIFECYCLE PROCESS

Because of the problems with the waterfall model discussed in [Section 4.2](#), the need for alternative SE processes led to the idea of agile SE. An agile lifecycle process (UX or SE) is a small-scope approach in which all lifecycle activities are performed for one or a small set of features of the product or system, and then the lifecycle is repeated for the next feature. An agile process is driven by needs formulated as user stories about capabilities instead of abstract system requirements and is characterized by small scope and fast deliveries of releases to get early usage-based feedback.

Because of the small scope, agile SE processes are fast in delivering a feature, very iterative, and responsive to change. This makes sense because agile means “nimble or responsive to change.”

### 4.3.1 Summary Comparison: Bookends on the Lifecycle Process Spectrum

The old waterfall model was an early attempt to organize software development processes. You performed each lifecycle activity for the whole system, then moved on to the next lifecycle activity. In contrast, an agile process offers a modern approach to achieve timely feedback. Here, you perform all the lifecycle activities for one feature of the system, then move on to the next feature.

### 4.3.2 Chunking to Reduce Scope and Elicit Quick Feedback

Chunking is a way of achieving small scope by breaking up the capabilities of the target system or consumer product and delivering those capabilities in incremental releases, typically in the order of priority defined by the client.

Fig. 4.2 provides a visual of how truly small the scope is in an agile approach (Beck, 1999). As the figure shows, the waterfall model makes one

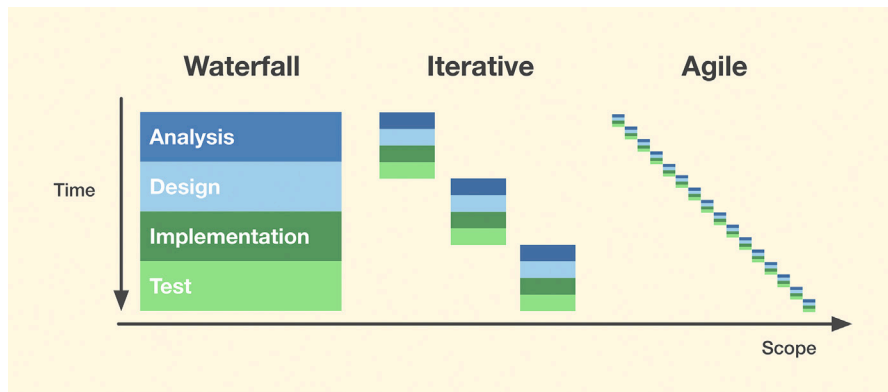


Fig. 4.2  
Scope and size of deliverables in the waterfall, iterative, and agile processes (adapted from Beck (1999), used with permission of IEEE).

big pass through the lifecycle activities for the whole system at one time. An iterative approach takes a step toward smaller but still fairly large chunks, resulting in multiple passes. The agile approach shows large numbers of frequent iterations for very small chunks of features of a product—a sign of true agility.

In his book on lean startups, Ries (2011) declares that agility is perhaps the single most important thing in successful product development. It is not so much about brilliant ideas, technology, far-seeing vision, good timing, or even luck. Instead, you should have a “process for adapting to situations as they reveal

themselves.” Lean UX (Gothelf & Seiden, 2021) is one variation of agile UX that focuses on producing a minimum viable product (MVP) in each sprint.

In an agile SE process, a sprint is an iteration associated with a release in which “a usable and potentially releasable product increment is created.”<sup>1</sup> In other words, “[e]ach sprint has a goal of what is to be built, a design and flexible plan that will guide building it, the work, and the resultant product increment.”

Agile SE processes deliver to the client and/or users meaningful small-scope product or system chunks of capabilities and working software features with every iteration. This gives the clients and users an opportunity to utilize these capabilities and provide immediate feedback. Hence, we never go too far down an unproductive path before we can make a course correction.

### 4.3.3 On the UX Side, There Has Always Been a Measure of Agility Without Chunking

These same characteristics—frequent iteration, feedback, learning, and responsiveness to change—have always been important in UX. In fact, the discipline of UX design was built on these foundational principles, so there was no need to do chunking to achieve them.

In the UX design process, there have been many ways of revealing situations that need attention. Users participate in ideation and sketching for design. We run all models and work products by the client. We do early and frequent design evaluations using low-fidelity prototypes, simulating the experience for the user and learning from it before building the real system.

### 4.3.4 Software Engineering Is Without the Luxury of Making User-Facing Prototypes

In a way, SE not having the luxury of making user-facing prototypes has been easier for the UX people. On the SE side of things, most of the concerns are about system aspects. Therefore, their artifacts are about representations of the inner workings of the system, which tend to be abstract and technical.

On the UX side of things, we are dealing with what the user sees and feels. Therefore, it is easy for us to give the user a flavor of a particular design concept by mocking up prototypes of these user-facing aspects. The SE people could not make meaningful low-fidelity mockups. To show something to users in the days prior to agile systems, they typically had to build the whole working system. To be fair, the SE side did experiment with rapid prototyping

---

<sup>1</sup><https://www.scrum.org/resources/what-is-a-sprint-in-scrum>

(Wasserman & Shewmake, 1982a, 1982b), but it was never really part of their mainstream process models.

### 4.3.5 SE Had No Interest in Users Anyway

Before agile processes, the SE people were not known for talking with users about envisioned behavior with just a sketch or mockup. They did not have touchpoints with users that focused on the product and experience side of things to situate the users in the design solution domain.

Rather, the SE people historically focused on the process side of things and technical issues, such as code structure, code understanding, and code reusability. If anything, they looked for ways to make it better for the programmer, not the end user.

Their “aha” moment arrived when they figured out that they could flip their approach to delivering chunks that people can use and give product-oriented feedback. Since then, agile<sup>2</sup> software development approaches have quickly become the de facto standard.

### 4.3.6 Why Does UX Follow SE in Becoming Agile?

There is a practical reason why UX designs still might have to be chunked for delivery to the SE people: to keep pace with sprints on the agile SE side. The agile SE people are the system builders. Because they use the agile approach, they will build it in chunks. So, as a discipline, agile UX has evolved to fit that model of development. We now deliver our UX design chunks to the client, users, product owners, and other stakeholders for direct feedback and, eventually, to the SE people for implementation.

See [Chapter 26](#) on connecting agile UX with agile SE for more about the characteristics of agile processes.

---

## 4.4 THE FUNNEL MODEL OF AGILE UX

We created what we call the funnel model of agile UX as a way to envision UX design activities before syncing them with agile SE sprints (for overall conceptual design) and after syncing with SE (for individual feature design). When we say “agile UX” in this book, we mean agile UX as set within this model.

---

<sup>2</sup>In the SE world, agile has often been granted a capital A (i.e., Agile), giving it special meaning. Because we do not make fine distinctions beyond the scope of this chapter, we will not deify the concept here.

### 4.4.1 Why Was a New Model Needed?

Agile UX became the way to bring design for the user experience to agile SE, but there were problems with the way agile UX was approached initially:

- Being agile was interpreted simply as going fast.
- The imitating of agile SE flow in sprints suffers from a fundamental mismatch with UX concerns.

***Speed kills: Rapidness and agility are not the same.*** Sometimes people confuse being agile with being fast. Although an agile process will often be rapid, agility is not defined by speed. Being rapid simply means working faster; being agile is about chunking for small-scope design and delivery so we can react to new lessons learned through usage.

Arnowitz (2013) cautions us that, while agile processes are almost always associated with speed, a single-minded focus on speed is almost guaranteed to be detrimental to the quality of the resulting user experience. Putting speed above everything else is a reckless response to pressure for rapid turnaround and is “proven to create Frankenstein UIs within a mere two to three iterations. That’s speed” (Arnowitz, 2013, p. 76). When you are driving a car and begin to get lost, driving faster will not help you find the way.

***The single biggest problem: UX was expected to emulate the agile SE flow completely.*** The SE world has turned agile, and the UX world has struggled to follow suit. Because many people thought agile UX workflow should mirror agile SE flow exactly to keep in sync with agile SE development, the UX teams began churning out chunks of UX design. A good UX design is holistic, cohesive, and self-consistent, however, and these new agile UX practitioners were just producing chunks without doing the upfront work necessary to establish a coherent overview or cohesive conceptual design. As they got into sync with the SE sprints, the design became fragmented.

Arnowitz (2013) says that the way agile SE processes are usually practiced, they are UX-hostile environments. Design is what we do in UX, so embodying the opposite of a holistic view in agile practice can easily promote fragmentation.

What was missing was a way for UX to do some kind of initial large-scope user and UX research and conceptual design before getting into a small-scope rhythm with SE. There was some literature about this problem, but it was mostly discussed as an exception or special case. In the funnel model of agile UX, (next section) we show how to include some upfront UX research (also see [Part 2](#)) and design in a mainstream view of agile UX.

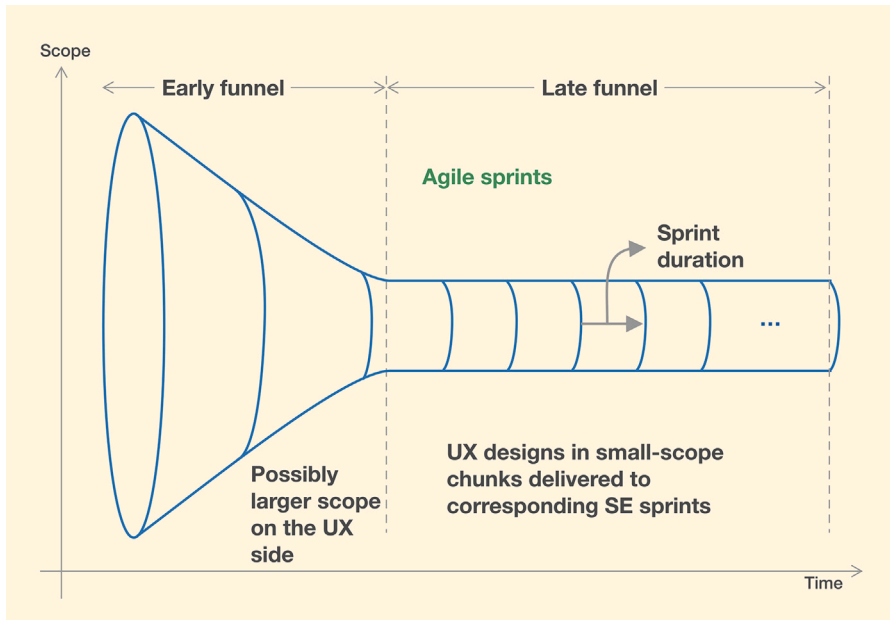


Fig. 4.3  
The funnel model of agile UX.

#### 4.4.2 Introducing the Funnel Model of Agile UX

The funnel model of agile UX (Fig. 4.3) has two major parts: the early funnel and the late funnel.

**Scope in the funnel model.** The vertical dimension of the diagram represents scope. A larger funnel diameter at any point on the funnel represents a larger scope there. Fig. 4.3 shows a typical case where the scope of the early funnel is larger than the scope of the late funnel.

**Speed and rigor in the funnel model.** The horizontal dimension of the diagram is time, representing how long activities in the funnel take to play out. The stripes or segments depicted on the funnel visually represent iterations or sprints, and the length of a segment represents the duration in time of that sprint and, by implication, the speed of methods and techniques that have to be used in that iteration. Longer sprints usually correspond with higher rigor, which will need methods and techniques that are more thorough and meticulous for that iteration (Fig. 4.4).

#### 4.4.3 Late-Funnel Activities

The late funnel (the spout on the right side of Fig. 4.3) is where the agile UX and agile SE processes are working in synchrony. Here, both the UX and SE

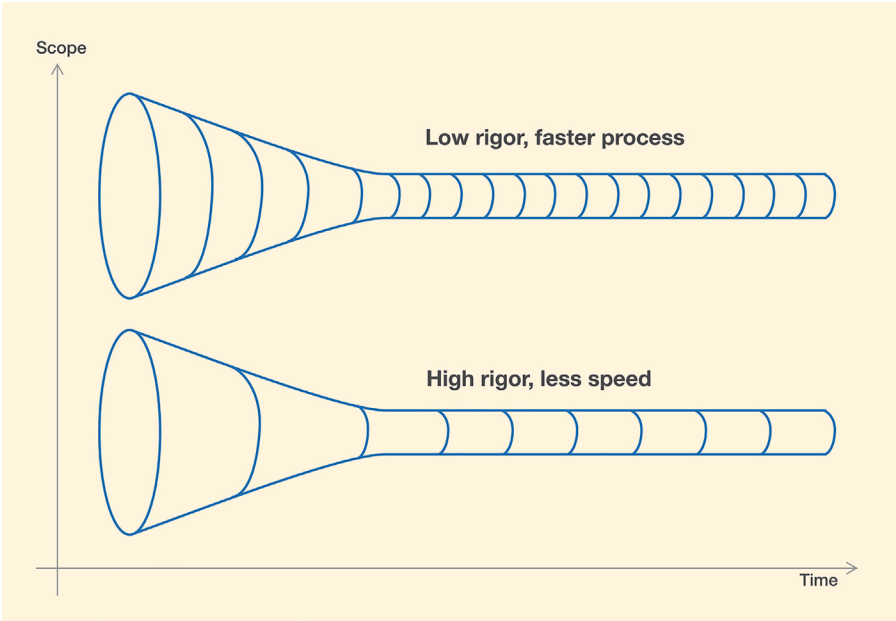


Fig. 4.4  
Low rigor and high speed  
versus high rigor and slow  
speed in the funnel model.

activities are typically described in terms of small chunks within a small scope (represented by the small diameter of the funnel spout) delivered within a relatively small time increment (narrow sprint duration stripe).

In theory, at least, each delivered software chunk is tested until it works and then integrated with the rest. The result of that is regression testing (looking

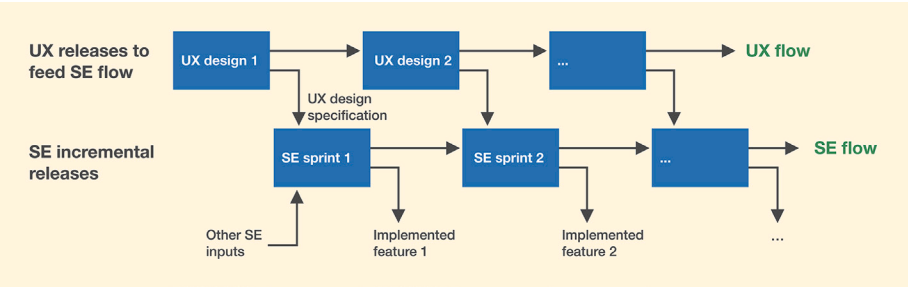
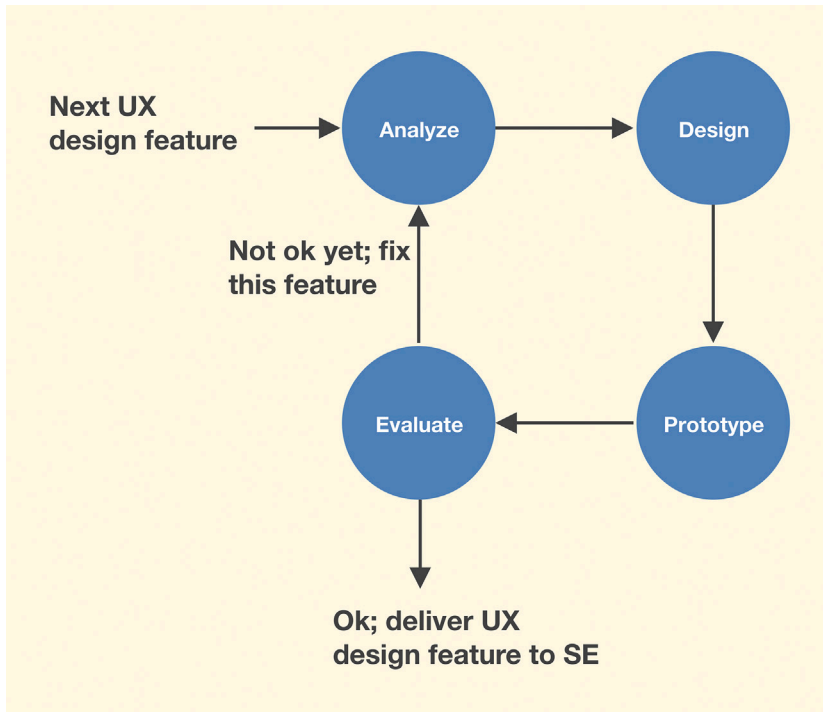


Fig. 4.5  
Syncing agile UX with agile  
SE sprints in the late-funnel  
flow.

back) until it all works. The idea is to begin and end each iteration with something that works (Constantine, 2002). Constant and close (but informal) communication produces continuous feedback.

**Syncing agile UX with agile SE sprints.** Once agile SE gets into the rhythm of going through sprints to produce implementations of a chunk of features, the idea of agile UX is to sync with an agile SE sprint by providing a UX design chunk (Fig. 4.5). The SE team then implements the chunk along with its design of the corresponding functionality in a sequence of sprints.



*Fig. 4.6*  
Mini agile UX lifecycle  
process within a sprint.

Each UX design box of Fig. 4.5 (e.g., UX design 1) contains a mini UX lifecycle, such as the one shown in Fig. 4.6.

We talk much more about how the agile UX and agile SE processes are coordinated in Chapter 26.

#### 4.4.4 Early-Funnel Activities

Meanwhile, before we can do the small-scope incremental releases in the synchronized late-funnel flow of Fig. 4.3, we have to start with some full-scope UX analysis and design upfront in the early funnel. A new UX design must start with a top-down view to understand the ecology and needs of the work practice and to establish a conceptual design, a high-level design vision that conveys the designer’s mental model of the design to users (see Chapter 14).

This is a requirement for UX because of the nature of UX design. Unlike code that is invisible to the user, UX design is not. Code is malleable in that it can be structured and refactored in every release. Redoing the interaction design like that will drive the users crazy.

This upfront UX activity, sometimes called “sprint 0” because it precedes the first sprint in the late funnel, puts “UX strategic design decisions upfront, where they belong” (Arnowitz, 2013, p. 78) to establish:

- An overview, a skeleton on which to put the features

- A solid coherent conceptual design to guide the design for the features
- An initial top-down design

This will entail as much of these upfront activities as necessary to understand needs and workflow:

- UX research data gathering ([Chapter 6](#))
- Analysis ([Chapter 7](#))
- Modeling ([Chapters 8 and 9](#))
- Conceptual design ([Chapter 14](#))

*The need to establish a conceptual design in the early funnel.* In any project charged with designing and building a brand-new system or a new version of a confusing existing system, you will need to establish a clear conceptual design upfront as scaffolding for a coherent overall UX design. A conceptual design is:

- A self-consistent high-level model or theme of how the system works
- A framework to aid users in acquiring their own mental model of how the system behaves
- By its nature, a large-scope notion

A small-scope approach at this point is likely to lead to a fragmented conceptual design. After the conceptual design is established, however, the UX team can move to a small scope in the late funnel to deliver incrementally the detailed UX designs of individual features to the SE side.

## Example: Developing a New Smartphone Design From Scratch

Suppose your project is to create a brand-new smartphone to compete with the current market leaders. It has been decided that this entails an entirely new and innovative conceptual design—a design that is better and more exciting to consumers than existing options offered in the market.

This case may require a significant large-scope effort in the early funnel to create a full conceptual design, to set the overall ecology of the smartphone, and to create a cohesive design upfront. It is simply not possible to design the overall operating model of a smartphone or any brand-new consumer-facing application without beginning with a large-scope design on the UX side.

*Small systems with low complexity.* Small systems with low complexity are an example that calls for large scope, but for a different reason: The system has low complexity, so the analysis and design (both conceptual and detailed) can be handled in a single shot.

Small systems with low complexity do not usually benefit much from a small-scope approach on the UX side. So, in fact, you can use a large scope through the whole funnel (picture the whole funnel being almost as wide as a wide mouth). If the system is not large, then there may not be enough of a pipeline or breadth of system features or complexity for the UX team to chunk the designs into small-scope increments. In that case, you can deliver it to the SE side in large scope and let them decompose it into features for their own small-scope implementation, if necessary.

***The nexus of early and late parts of the funnel.*** The transition to the late funnel is a crucial point in a project; it is the beginning of so-called crunch time, where the UX people get in gear and sync with the agile SE people. Now is the time for user or customer stories (see [Section 26.7.1.2](#)) to drive your design iterations, but the deep understanding (in total) you developed in UX research and modeling informs that design.

---

## 4.5 SEGUE

In this chapter, we discussed the agile approach to UX design and how it syncs with the counterpart SE lifecycle. Now we are ready to dive into the process chapters. In the next chapter, as a prelude, we first talk about practical considerations, such as resources (e.g., a design studio) and personnel (e.g., UX roles) and how projects get started. We introduce an example in the next chapter that we will use for the rest of the process chapters.